VAC LAB CYCLE

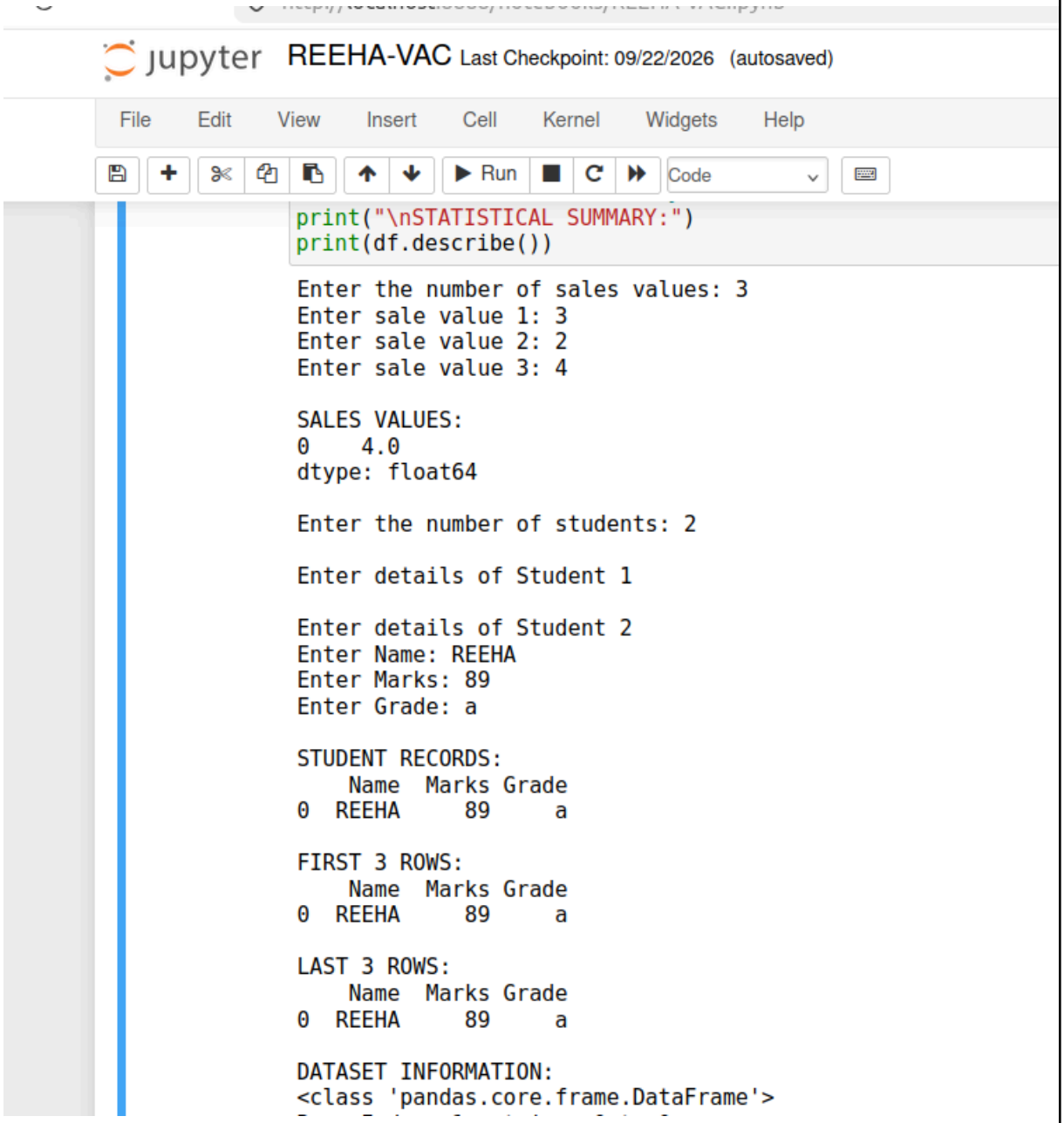
SUBMITTED BY:
REEHA SHAKEER
II BCA

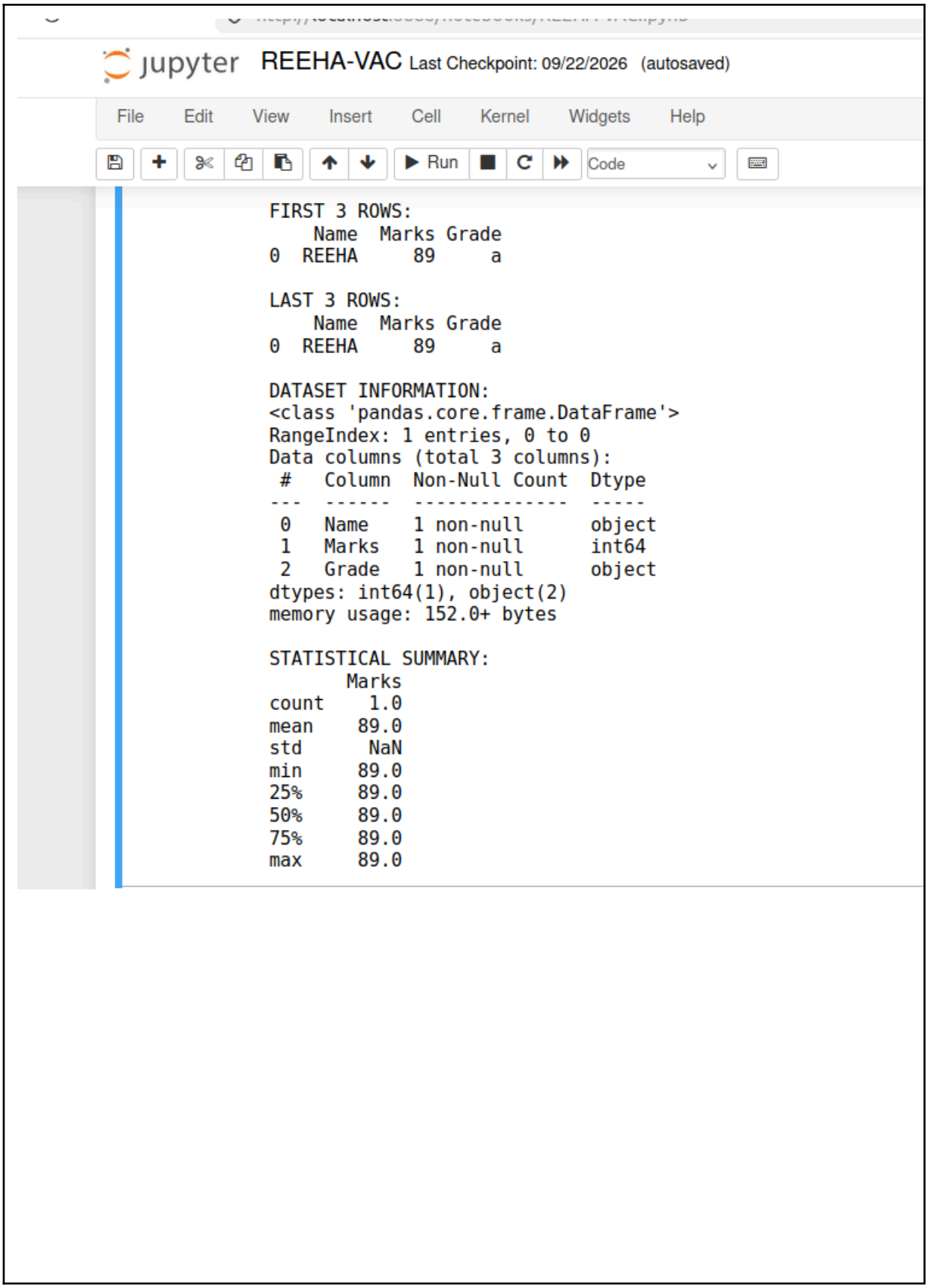
AIM:
Create and display a series containing sales values.Create a DataFrame for students records and store name,marks and grade in a DataFrame.
Display First and Last rows of a Dataset,Display Dataset Information.

PROGRAM:

```
import pandas as pd
# Create a Series using user input
n = int(input("Enter the number of sales values: "))
sales = []
for i in range(n):
value = float(input(f"Enter sale value {i + 1}: "))
sales.append(value)
sales_series = pd.Series(sales)
print("\nSALES VALUES:")
print(sales_series)
# Create Student DataFrame using user input
n = int(input("\nEnter the number of students: "))
names = []
marks = []
grades = []
for i in range(n):
print(f"\nEnter details of Student {i + 1}")
name = input("Enter Name: ")
mark = int(input("Enter Marks: "))
grade = input("Enter Grade: ")
names.append(name)
marks.append(mark)
grades.append(grade)
student_records = {
'Name': names,
'Marks': marks,
'Grade': grades
}
df = pd.DataFrame(student_records)
print("\nSTUDENT RECORDS:")
print(df)
# Display first rows
print("\nFIRST 3 ROWS:")
print(df.head(3))
# Display last rows
print("\nLAST 3 ROWS:")
print(df.tail(3))
# Display dataset information
print("\nDATASET INFORMATION:")
df.info()
# Generate statistical summary
print("\nSTATISTICAL SUMMARY:")
print(df.describe())
```

OUTPUT:


jupyter REEHA-VAC Last Checkpoint: 09/22/2026 (autosaved)
File Edit View Insert Cell Kernel Widgets Help
Run
Code
print("\nSTATISTICAL SUMMARY:")
print(df.describe())
Enter the number of sales values: 3
Enter sale value 1: 3
Enter sale value 2: 2
Enter sale value 3: 4
SALES VALUES:
0 4.0
dtype: float64
Enter the number of students: 2
Enter details of Student 1
Enter details of Student 2
Enter Name: REEHA
Enter Marks: 89
Enter Grade: a
STUDENT RECORDS:
Name Marks Grade
0 REEHA 89 a
FIRST 3 ROWS:
Name Marks Grade
0 REEHA 89 a
LAST 3 ROWS:
Name Marks Grade
0 REEHA 89 a
DATASET INFORMATION:
<class 'pandas.core.frame.DataFrame'>

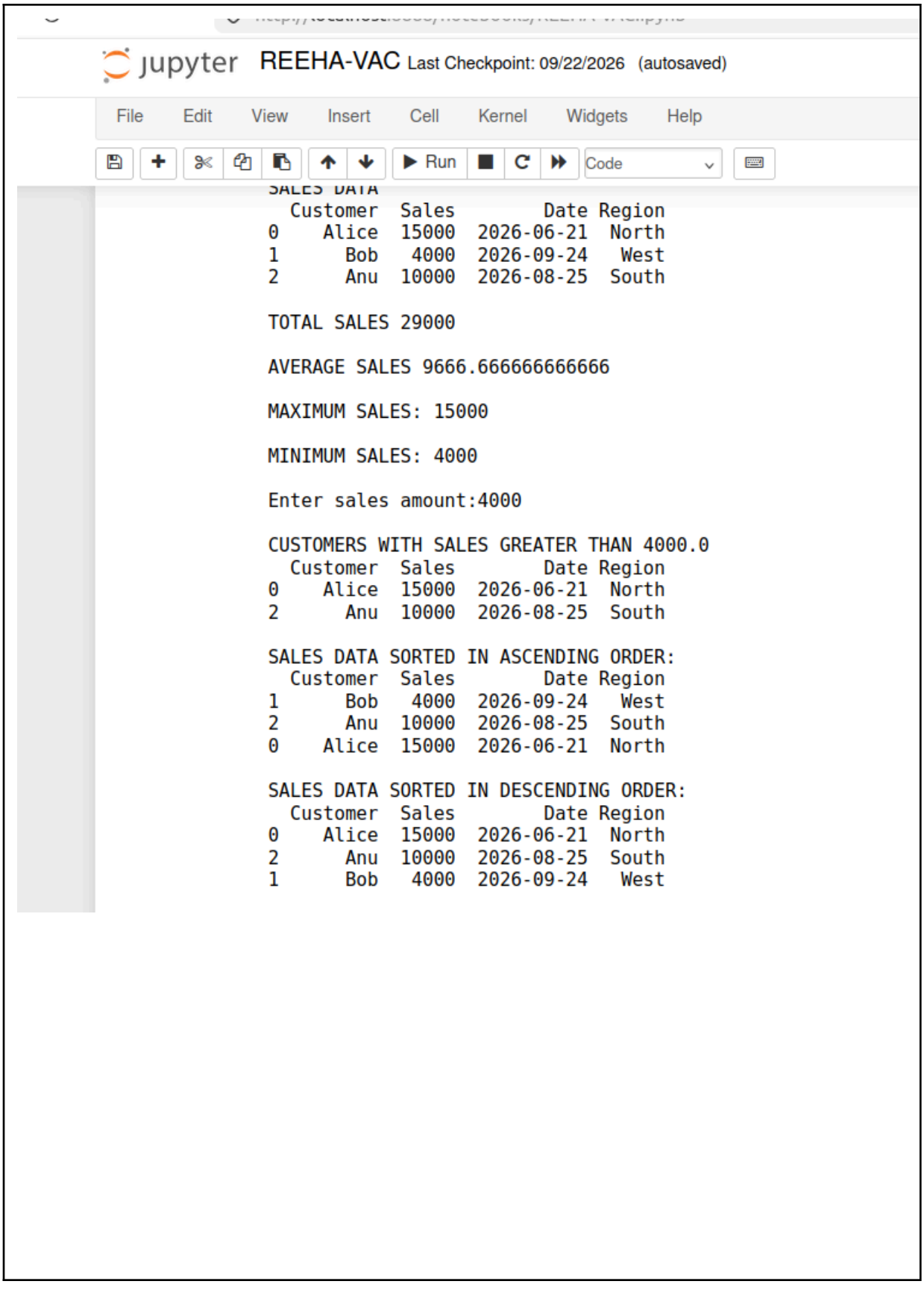

jupyter REEHA-VAC Last Checkpoint: 09/22/2026 (autosaved)
File Edit View Insert Cell Kernel Widgets Help
Run
Code
FIRST 3 ROWS:
Name Marks Grade
0 REEHA 89 a
LAST 3 ROWS:
Name Marks Grade
0 REEHA 89 a
DATASET INFORMATION:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1 entries, 0 to 0
Data columns (total 3 columns):
Column Non-Null Count Dtype
--- ------ -------------- -----
0 Name 1 non-null object
1 Marks 1 non-null int64
2 Grade 1 non-null object
dtypes: int64(1), object(2)
memory usage: 152.0+ bytes
STATISTICAL SUMMARY:
Marks
count 1.0
mean 89.0
std NaN
min 89.0
25% 89.0
50% 89.0
75% 89.0
max 89.0

AIM:
Load sales dataset from csv file / excel file. Find total sales, average sales, maximum and minimum sales. Display customers with sales greater than a specific amount. Sort sales data.

PROGRAM:

```
import pandas as pd
df = pd.read_csv("sales.csv")
print("SALES DATA")
print(df)
total_sales = df["Sales"].sum()
print("\nTOTAL SALES", total_sales)
average_sales = df["Sales"].mean()
print("\nAVERAGE SALES", average_sales)
maximum_sales = df["Sales"].max()
print("\nMAXIMUM SALES:", maximum_sales)
minimum_sales = df["Sales"].min()
print("\nMINIMUM SALES:", minimum_sales)
amount = float(input("\nEnter sales amount:"))
filtered_data = df[df["Sales"] > amount]
print("\nCUSTOMERS WITH SALES GREATER THAN", amount)
print(filtered_data)
print("\nSALES DATA SORTED IN ASCENDING ORDER:")
print(df.sort_values(by="Sales"))
print("\nSALES DATA SORTED IN DESCENDING ORDER:")
print(df.sort_values(by="Sales", ascending=False))
```

OUTPUT:

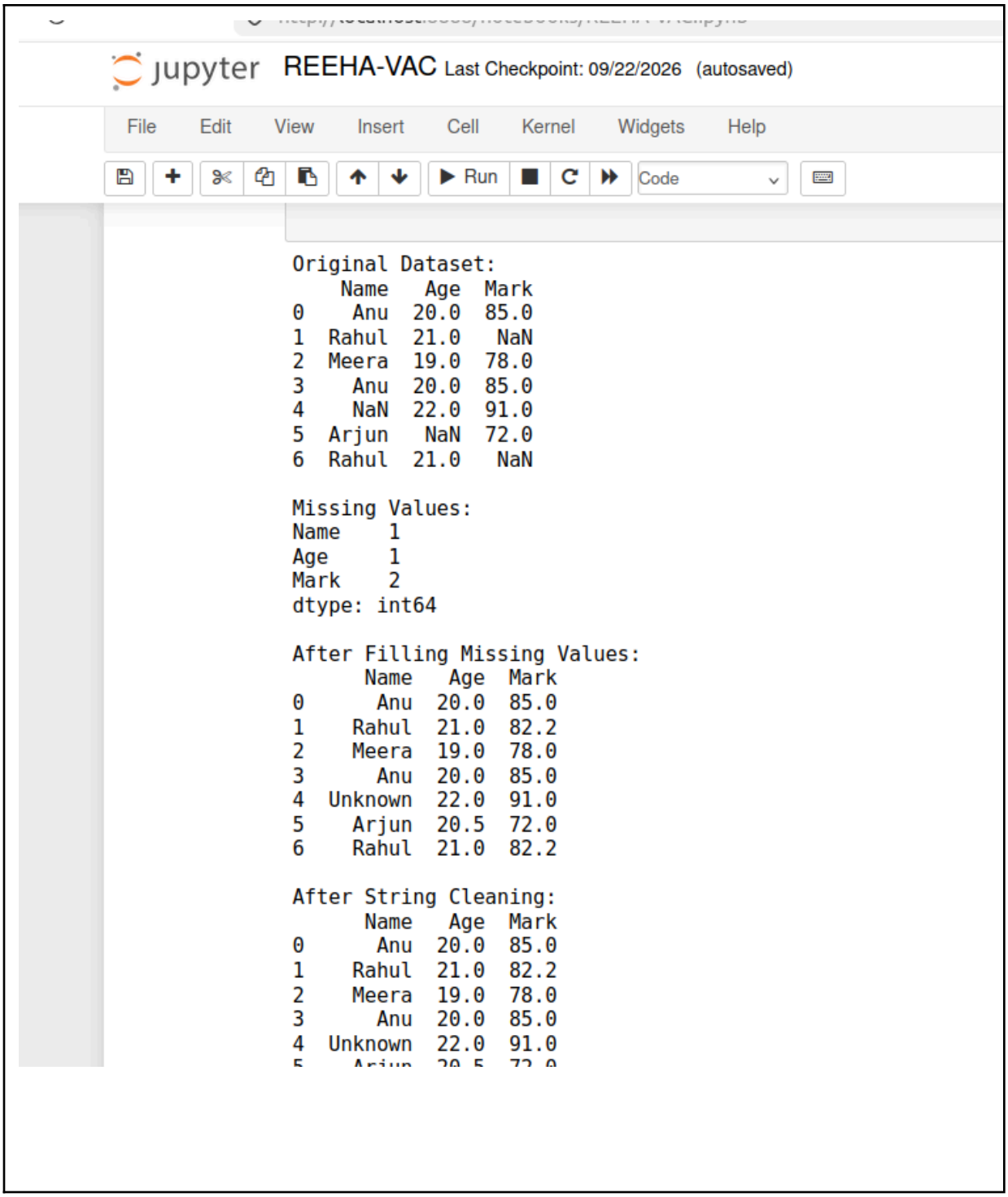

jupyter REEHA-VAC Last Checkpoint: 09/22/2026 (autosaved)
File Edit View Insert Cell Kernel Widgets Help
Run
Code
SALES DATA
Customer Sales Date Region
0 Alice 15000 2026-06-21 North
1 Bob 4000 2026-09-24 West
2 Anu 10000 2026-08-25 South
TOTAL SALES 29000
AVERAGE SALES 9666.666666666666
MAXIMUM SALES: 15000
MINIMUM SALES: 4000
Enter sales amount:4000
CUSTOMERS WITH SALES GREATER THAN 4000.0
Customer Sales Date Region
0 Alice 15000 2026-06-21 North
2 Anu 10000 2026-08-25 South
SALES DATA SORTED IN ASCENDING ORDER:
Customer Sales Date Region
1 Bob 4000 2026-09-24 West
2 Anu 10000 2026-08-25 South
0 Alice 15000 2026-06-21 North
SALES DATA SORTED IN DESCENDING ORDER:
Customer Sales Date Region
0 Alice 15000 2026-06-21 North
2 Anu 10000 2026-08-25 South
1 Bob 4000 2026-09-24 West

AIM:
Load dataset, detect Missing Values in Dataset, Fill Missing Values, Remove Missing Records, Detect Duplicate Records, Remove Duplicate Records, Correct Data Types, Perform String Cleaning Operations.

PROGRAM:

```
import pandas as pd
df = pd.read_csv("students.csv")
print("Original Dataset:")
print(df)
print("\nMissing Values:")
print(df.isnull().sum())
df["Age"] = df["Age"].fillna(df["Age"].mean())
df["Mark"] = df["Mark"].fillna(df["Mark"].mean())
df["Name"] = df["Name"].fillna("Unknown")
print("\nAfter Filling Missing Values:")
print(df)
df["Name"] = df["Name"].str.strip()
df["Name"] = df["Name"].str.title()
print("\nAfter String Cleaning:")
print(df)
print("\nDuplicate Records:")
print(df[df.duplicated()])
df = df.drop_duplicates()
print("\nAfter Removing Duplicate Records:")
print(df)
df["Age"] = df["Age"].astype(int)
df["Mark"] = df["Mark"].astype(float)
print("\nCorrected Data Types:")
print(df.dtypes)
df = df.dropna()
print("\nFinal Cleaned Dataset:")
print(df)
```

OUTPUT:


jupyter REEHA-VAC Last Checkpoint: 09/22/2026 (autosaved)
File Edit View Insert Cell Kernel Widgets Help
Run
Code
Original Dataset:
Name Age Mark
0 Anu 20.0 85.0
1 Rahul 21.0 NaN
2 Meera 19.0 78.0
3 Anu 20.0 85.0
4 NaN 22.0 91.0
5 Arjun NaN 72.0
6 Rahul 21.0 NaN
Missing Values:
Name 1
Age 1
Mark 2
dtype: int64
After Filling Missing Values:
Name Age Mark
0 Anu 20.0 85.0
1 Rahul 21.0 82.2
2 Meera 19.0 78.0
3 Anu 20.0 85.0
4 Unknown 22.0 91.0
5 Arjun 20.5 72.0
6 Rahul 21.0 82.2
After String Cleaning:
Name Age Mark
0 Anu 20.0 85.0
1 Rahul 21.0 82.2
2 Meera 19.0 78.0
3 Anu 20.0 85.0
4 Unknown 22.0 91.0

jupyter REEHA-VAC Last Checkpoint: 09/22/2026 (autosaved)

File Edit View Insert Cell Kernel Widgets Help

Run Code

```
      Name   Age  Mark
0      Anu  20.0  85.0
1    Rahul  21.0  82.2
2    Meera  19.0  78.0
3      Anu  20.0  85.0
4  Unknown  22.0  91.0
5    Arjun  20.5  72.0
6    Rahul  21.0  82.2

Duplicate Records:
    Name   Age  Mark
3    Anu  20.0  85.0
6  Rahul  21.0  82.2

After Removing Duplicate Records:
      Name   Age  Mark
0      Anu  20.0  85.0
1    Rahul  21.0  82.2
2    Meera  19.0  78.0
4  Unknown  22.0  91.0
5    Arjun  20.5  72.0

Corrected Data Types:
Name     object
Age       int64
Mark    float64
dtype: object

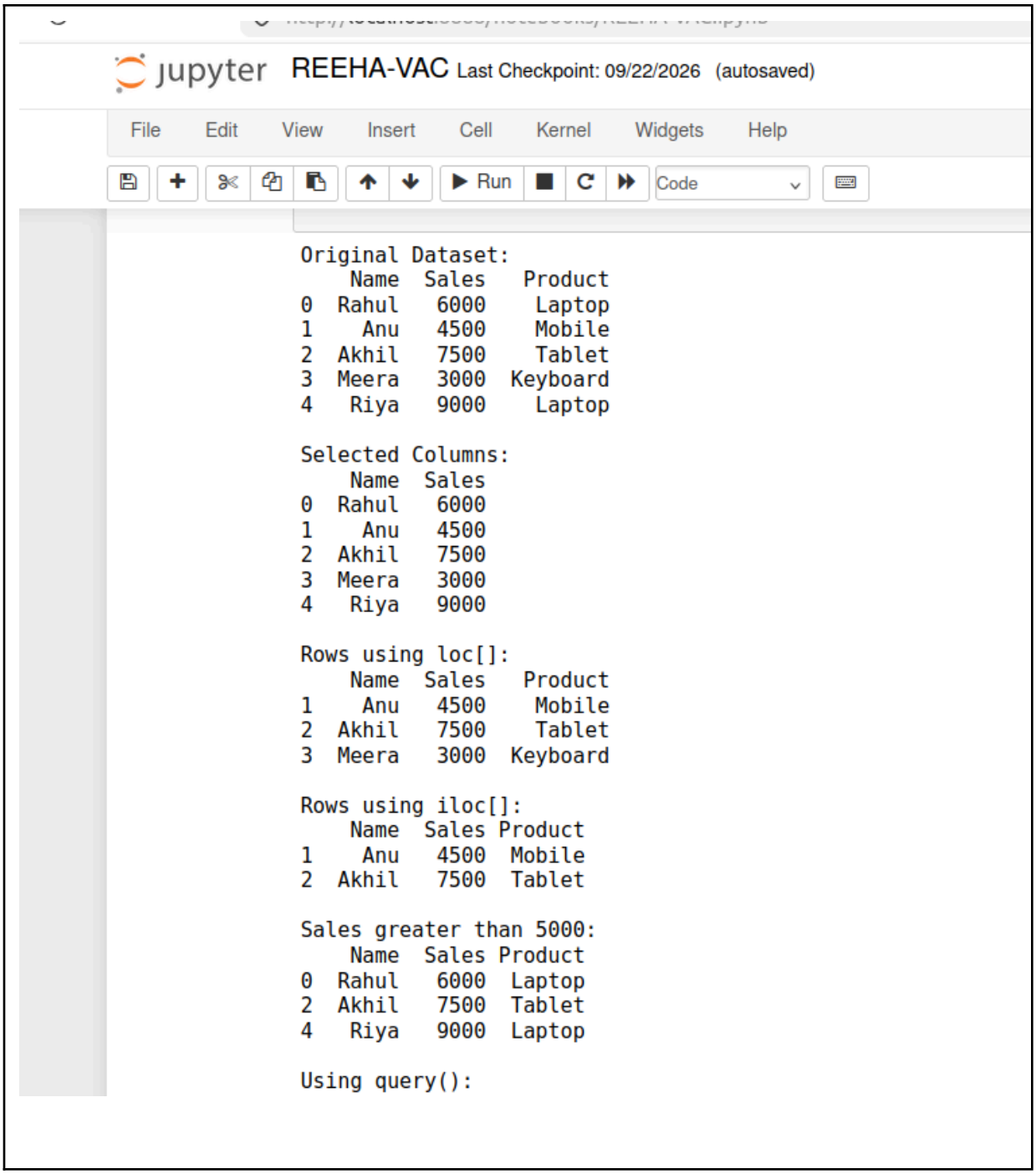
Final Cleaned Dataset:
      Name  Age  Mark
0      Anu   20  85.0
1    Rahul   21  82.2
2    Meera   19  78.0
4  Unknown   22  91.0
5    Arjun   20  72.0
```

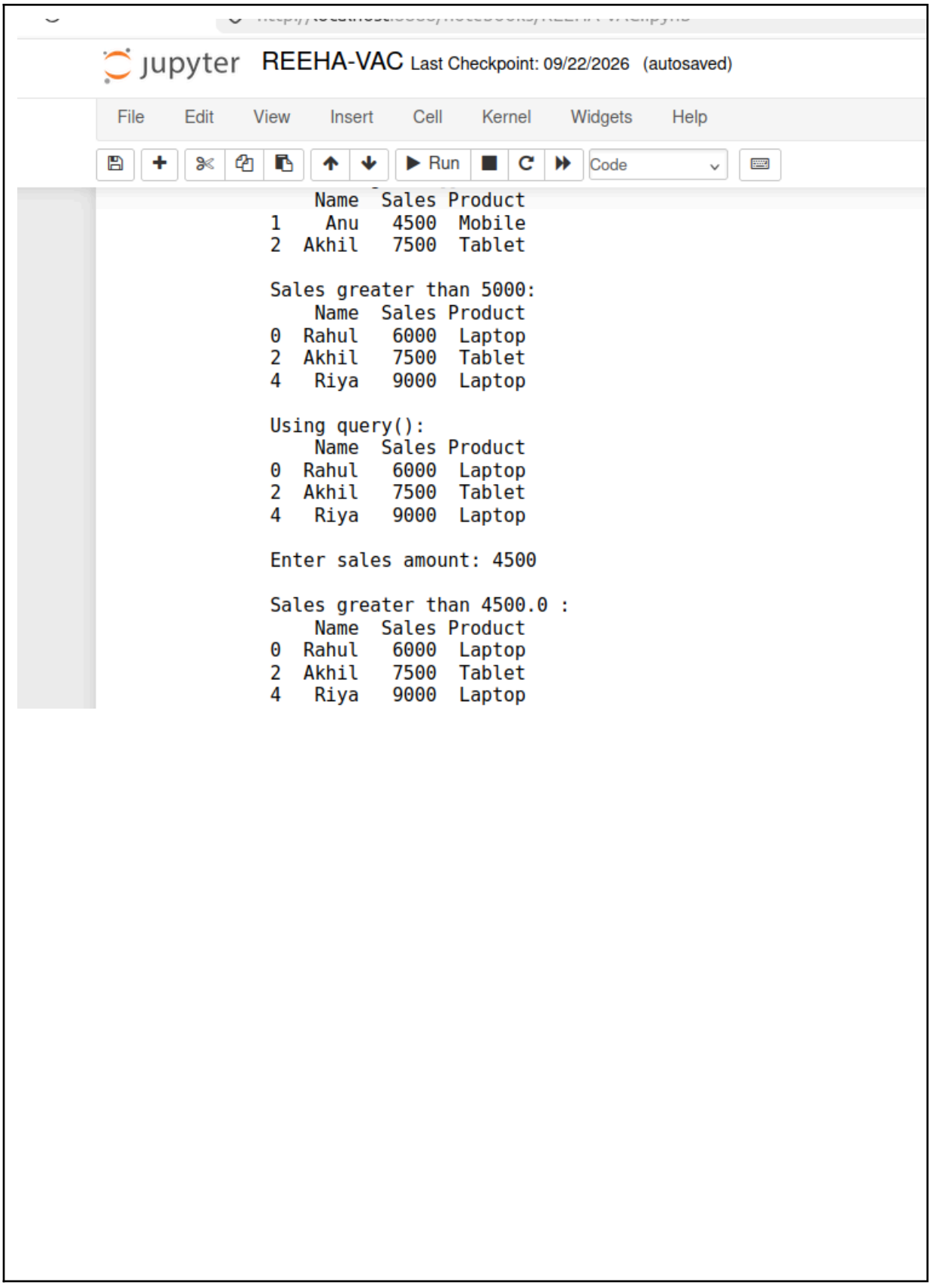
AIM:
Load dataset, Select Specific Columns from Dataset, Select Rows using loc[] and iloc[], Filter Dataset using Conditions, Query Dataset using query(), Filter sales greater than a specified value.

PROGRAM:

```
import pandas as pd
data={
"NAME":["SHAHAZA","HUSNA","FIYYAH","REEHA","RASMINA"],
"SALES":[60000,80000,45000,30000,70000],
"PRODUCT":["LAPTOP","MOBILE","TABLET","KEYBOARD","LAPTOP"]
}
df=pd.DataFrame(data)
df.to_csv("sales.csv",index=False)
df=pd.read_csv("sales.csv")
print("ORIGINAL DATASET:")
print(df)
print("\nSELECTED COLUMNS:")
print(df[["NAME","SALES"]])
print("\nROWS USING LOC[]")
print(df.loc[1:3])
print("\nROWS USING ILOC[]")
print(df.iloc[1:3])
print("\nSALES GREATER THAN 50000:")
print(df[df["SALES"]>50000])
print("\nUSING QUERY():")
print(df.query("SALES>50000"))
amount=float(input("\nENTER SALES AMOUNT:"))
print("\nSALES GREATER THAN",amount,":")
print(df[df["SALES"]>amount])
```

OUTPUT:


jupyter REEHA-VAC Last Checkpoint: 09/22/2026 (autosaved)
File Edit View Insert Cell Kernel Widgets Help
Run
Code
Original Dataset:
Name Sales Product
0 Rahul 6000 Laptop
1 Anu 4500 Mobile
2 Akhil 7500 Tablet
3 Meera 3000 Keyboard
4 Riya 9000 Laptop
Selected Columns:
Name Sales
0 Rahul 6000
1 Anu 4500
2 Akhil 7500
3 Meera 3000
4 Riya 9000
Rows using loc[]:
Name Sales Product
1 Anu 4500 Mobile
2 Akhil 7500 Tablet
3 Meera 3000 Keyboard
Rows using iloc[]:
Name Sales Product
1 Anu 4500 Mobile
2 Akhil 7500 Tablet
Sales greater than 5000:
Name Sales Product
0 Rahul 6000 Laptop
2 Akhil 7500 Tablet
4 Riya 9000 Laptop
Using query():

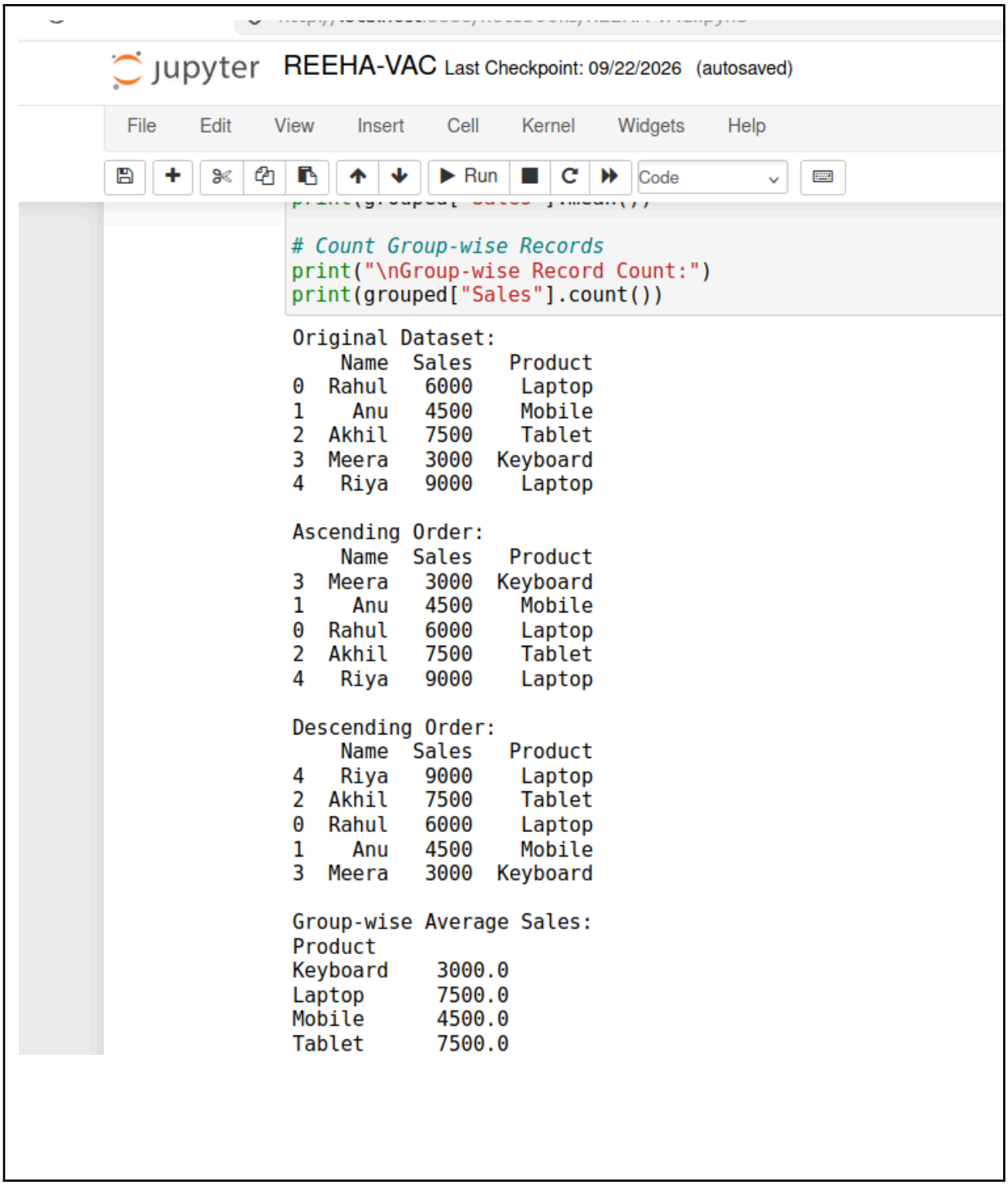

jupyter REEHA-VAC Last Checkpoint: 09/22/2026 (autosaved)
File Edit View Insert Cell Kernel Widgets Help
Run
Code
Name Sales Product
1 Anu 4500 Mobile
2 Akhil 7500 Tablet
Sales greater than 5000:
Name Sales Product
0 Rahul 6000 Laptop
2 Akhil 7500 Tablet
4 Riya 9000 Laptop
Using query():
Name Sales Product
0 Rahul 6000 Laptop
2 Akhil 7500 Tablet
4 Riya 9000 Laptop
Enter sales amount: 4500
Sales greater than 4500.0 :
Name Sales Product
0 Rahul 6000 Laptop
2 Akhil 7500 Tablet
4 Riya 9000 Laptop

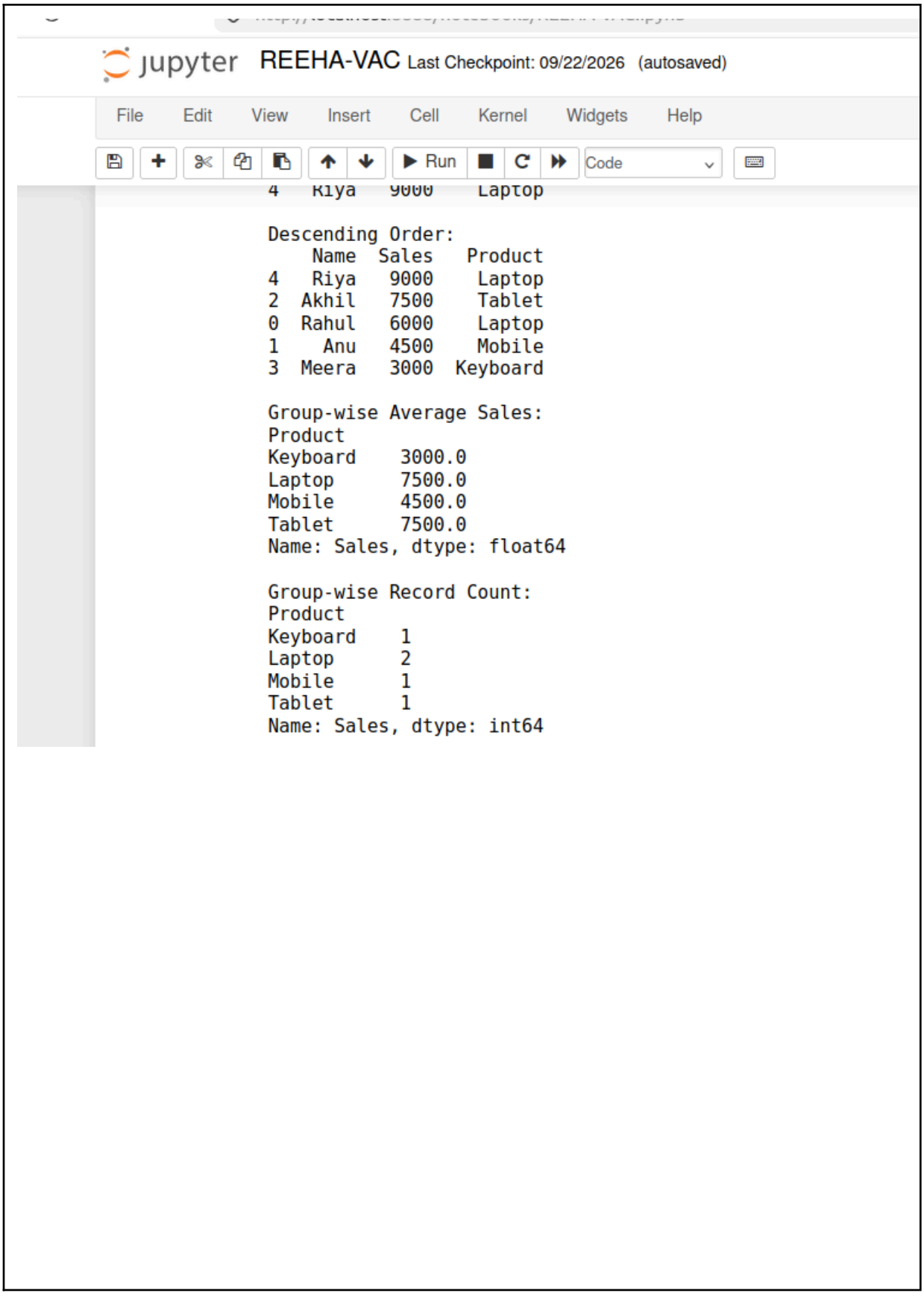
AIM:
Load dataset, Sort Dataset in Ascending Order, Sort Dataset in Descending Order, Group Dataset, Calculate Group-wise Average, Count Group-wise Records.

PROGRAM:

```
import pandas as pd
data = {
"Name": ["Rahul", "Anu", "Akhil", "Meera", "Riya"],
"Sales": [6000, 4500, 7500, 3000, 9000],
"Product": ["Laptop", "Mobile", "Tablet", "Keyboard", "Laptop"]
}
df = pd.DataFrame(data)
df.to_csv("sales.csv", index=False)
df = pd.read_csv("sales.csv")
print("Original Dataset:")
print(df)
print("\nAscending Order:")
print(df.sort_values("Sales"))
print("\nDescending Order:")
print(df.sort_values("Sales", ascending=False))
grouped = df.groupby("Product")
print("\nGroup-wise Average Sales:")
print(grouped["Sales"].mean())
print("\nGroup-wise Record Count:")
print(grouped["Sales"].count())
```

OUTPUT:


jupyter REEHA-VAC Last Checkpoint: 09/22/2026 (autosaved)
File Edit View Insert Cell Kernel Widgets Help
Run
Code
Count Group-wise Records
print("\nGroup-wise Record Count:")
print(grouped["Sales"].count())
Original Dataset:
Name Sales Product
0 Rahul 6000 Laptop
1 Anu 4500 Mobile
2 Akhil 7500 Tablet
3 Meera 3000 Keyboard
4 Riya 9000 Laptop
Ascending Order:
Name Sales Product
3 Meera 3000 Keyboard
1 Anu 4500 Mobile
0 Rahul 6000 Laptop
2 Akhil 7500 Tablet
4 Riya 9000 Laptop
Descending Order:
Name Sales Product
4 Riya 9000 Laptop
2 Akhil 7500 Tablet
0 Rahul 6000 Laptop
1 Anu 4500 Mobile
3 Meera 3000 Keyboard
Group-wise Average Sales:
Product
Keyboard 3000.0
Laptop 7500.0
Mobile 4500.0
Tablet 7500.0


jupyter REEHA-VAC Last Checkpoint: 09/22/2026 (autosaved)
File Edit View Insert Cell Kernel Widgets Help
Run
Code
4 Riya 9000 Laptop
Descending Order:
Name Sales Product
4 Riya 9000 Laptop
2 Akhil 7500 Tablet
0 Rahul 6000 Laptop
1 Anu 4500 Mobile
3 Meera 3000 Keyboard
Group-wise Average Sales:
Product
Keyboard 3000.0
Laptop 7500.0
Mobile 4500.0
Tablet 7500.0
Name: Sales, dtype: float64
Group-wise Record Count:
Product
Keyboard 1
Laptop 2
Mobile 1
Tablet 1
Name: Sales, dtype: int64

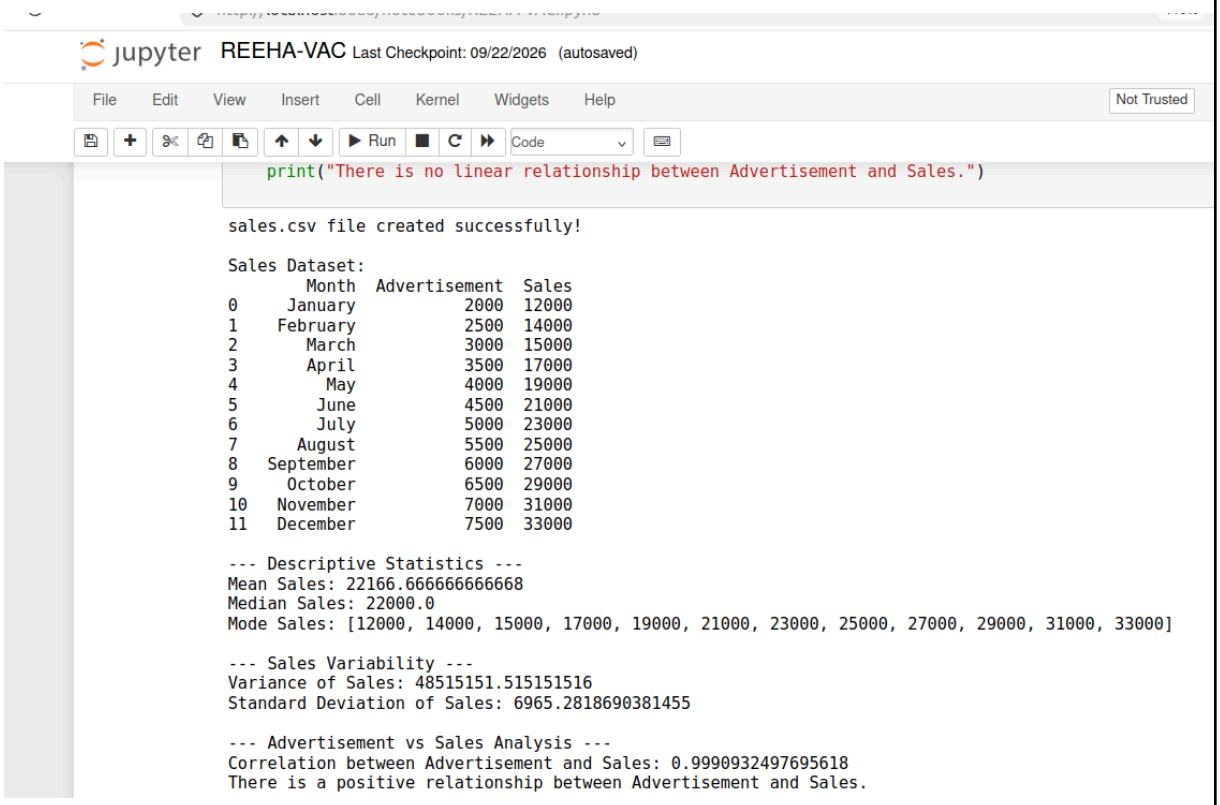
AIM:
Load sales dataset, Calculate descriptive statistics like Mean, Median, and Mode, Study sales variability using Variance and Standard Deviation, Find Correlation between Columns: Advertisement vs sales analysis.

PROGRAM:

```
import pandas as pd
data = {
"Month": [
"January", "February", "March", "April",
"May", "June", "July", "August",
"September", "October", "November", "December"
],
"Advertisement": [
2000, 2500, 3000, 3500,
4000, 4500, 5000, 5500,
6000, 6500, 7000, 7500
],
"Sales": [
12000, 14000, 15000, 17000,
19000, 21000, 23000, 25000,
27000, 29000, 31000, 33000
]
}
df = pd.DataFrame(data)
df.to_csv("sales.csv", index=False)
print("sales.csv file created successfully!")
df = pd.read_csv("sales.csv")
print("\nSales Dataset:")
print(df)
print("\n--- Descriptive Statistics ---")
mean_sales = df["Sales"].mean()
median_sales = df["Sales"].median()
mode_sales = df["Sales"].mode()
print("Mean Sales:", mean_sales)
print("Median Sales:", median_sales)
if len(mode_sales) > 0:
print("Mode Sales:", list(mode_sales))
else:
print("No mode found")
print("\n--- Sales Variability ---")
variance_sales = df["Sales"].var()
std_sales = df["Sales"].std()
print("Variance of Sales:", variance_sales)
print("Standard Deviation of Sales:", std_sales)
print("\n--- Advertisement vs Sales Analysis ---")
correlation = df["Advertisement"].corr(df["Sales"])
print("Correlation between Advertisement and Sales:",
correlation)
if correlation > 0:
print("There is a positive relationship between Advertisement and Sales.")
```

```
elif correlation < 0:
print("There is a negative relationship between Advertisement and Sales.")
else:
print("There is no linear relationship between Advertisement and Sales.")
```

OUTPUT:



AIM:
Load dataset, handle missing values and duplicates and create Line Chart using Pandas for Monthly sales trend analysis. Create Bar Chart using Pandas for Product-wise sales comparison.
PROGRAM:

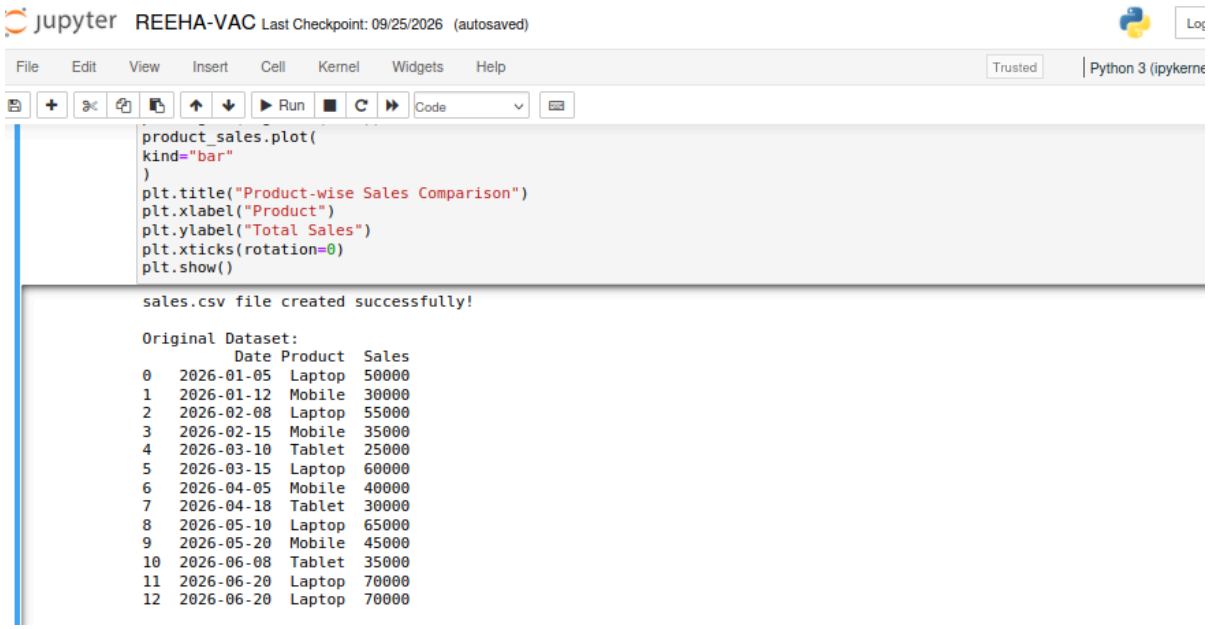
```
import pandas as pd
import matplotlib.pyplot as plt
data = {
"Date": [
"2026-01-05",
"2026-01-12",
"2026-02-08",
"2026-02-15",
"2026-03-10",
"2026-03-15",
"2026-04-05",
"2026-04-18",
"2026-05-10",
"2026-05-20",
"2026-06-08",
"2026-06-20",
"2026-06-20"
],
"Product": [
"Laptop",
"Mobile",
"Laptop",
"Mobile",
"Tablet",
"Laptop",
"Mobile",
"Tablet",
"Laptop",
"Mobile",
"Tablet",
"Laptop",
"Laptop"
],
"Sales": [
50000,
30000,
55000,
35000,
25000,
60000,
40000,
30000,
65000,
```

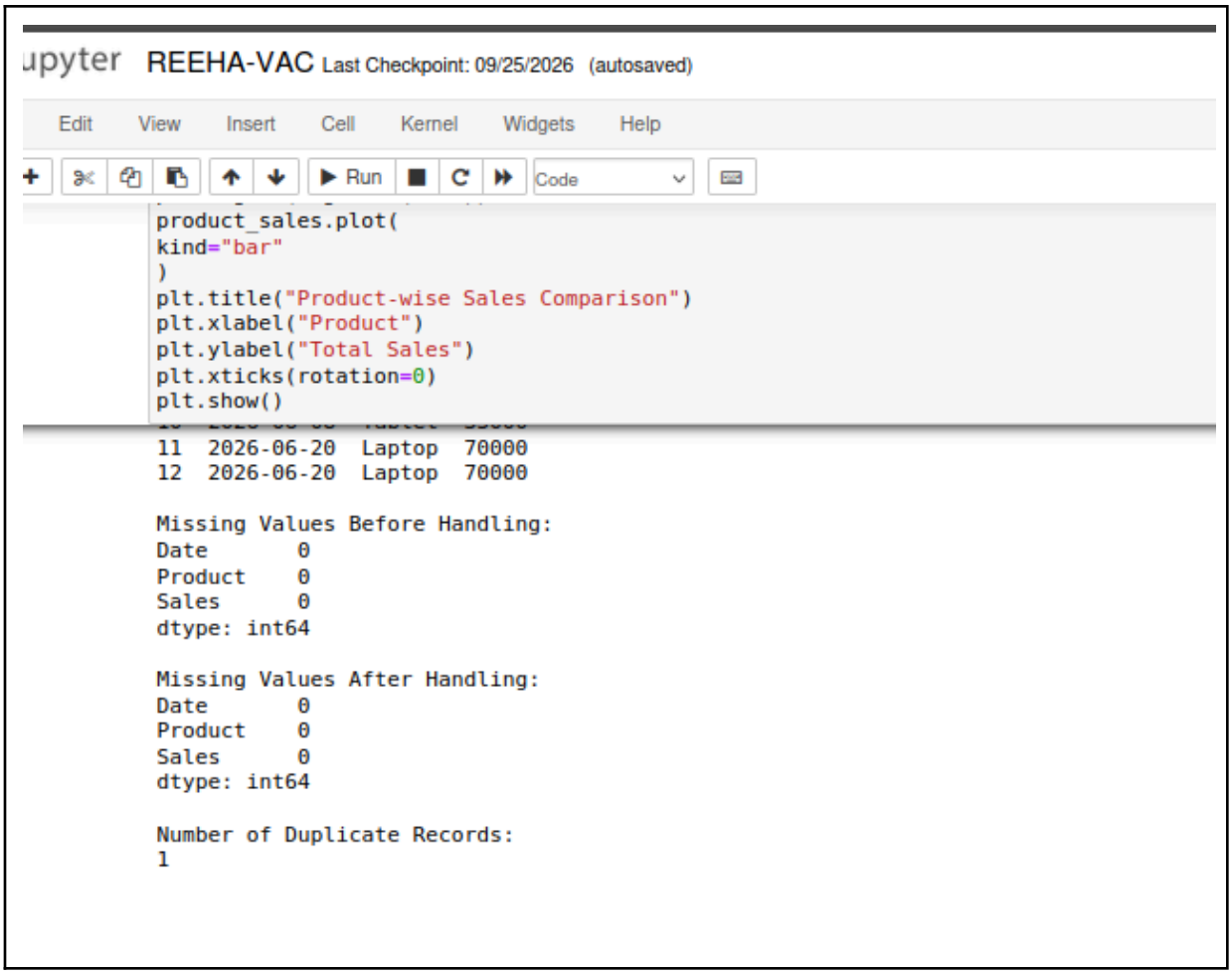
```
45000,
35000,

70000,
70000
]
}
df = pd.DataFrame(data)
df.to_csv("sales.csv", index=False)
print("sales.csv file created successfully!")
df = pd.read_csv("sales.csv")
print("\nOriginal Dataset:")
print(df)
print("\nMissing Values Before Handling:")
print(df.isnull().sum())
df["Product"] = df["Product"].fillna("Unknown")
df["Sales"] = df["Sales"].fillna(df["Sales"].mean())
df["Date"] = df["Date"].ffill()
print("\nMissing Values After Handling:")
print(df.isnull().sum())
print("\nNumber of Duplicate Records:")
print(df.duplicated().sum())
df = df.drop_duplicates()
print("\nDataset After Removing Duplicates:")
print(df)
df["Date"] = pd.to_datetime(df["Date"])
df["Month"] = df["Date"].dt.to_period("M").astype(str)
monthly_sales = df.groupby("Month")["Sales"].sum()
print("\nMonthly Sales:")
print(monthly_sales)
plt.figure(figsize=(8, 5))
monthly_sales.plot(
kind="line",
marker="o"
)
plt.title("Monthly Sales Trend")
plt.xlabel("Month")
plt.ylabel("Total Sales")
plt.grid(True)
plt.show()
product_sales = df.groupby("Product")["Sales"].sum()
print("\nProduct-wise Sales:")
print(product_sales)
plt.figure(figsize=(8, 5))
product_sales.plot(
kind="bar"
)
plt.title("Product-wise Sales Comparison")
plt.xlabel("Product")
plt.ylabel("Total Sales")
```

```
plt.xticks(rotation=0)
plt.show()
```

OUTPUT:




upyter REEHA-VAC Last Checkpoint: 09/25/2026 (autosaved)
Edit View Insert Cell Kernel Widgets Help
Run
Code
product_sales.plot(
kind="bar"
)
plt.title("Product-wise Sales Comparison")
plt.xlabel("Product")
plt.ylabel("Total Sales")
plt.xticks(rotation=0)
plt.show()
11 2026-06-20 Laptop 70000
12 2026-06-20 Laptop 70000
Missing Values Before Handling:
Date 0
Product 0
Sales 0
dtype: int64
Missing Values After Handling:
Date 0
Product 0
Sales 0
dtype: int64
Number of Duplicate Records:
1

upyter REEHA-VAC Last Checkpoint: 09/25/2026 (autosaved)

Edit View Insert Cell Kernel Widgets Help

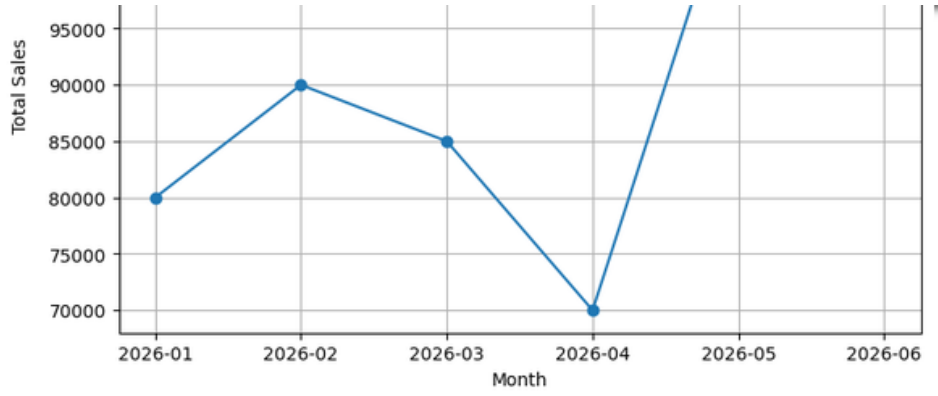
Run Code

```
product_sales.plot(
kind="bar"
)
plt.title("Product-wise Sales Comparison")
plt.xlabel("Product")
plt.ylabel("Total Sales")
plt.xticks(rotation=0)
plt.show()
```

```
10  2026-06-08  Tablet  35000
11  2026-06-20  Laptop  70000

Monthly Sales:
Month
2026-01     80000
2026-02     90000
2026-03     85000
2026-04     70000
2026-05    110000
2026-06    105000
Name: Sales, dtype: int64
```



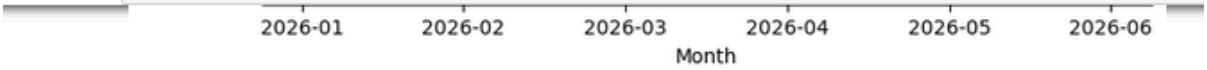


upyter REEHA-VAC Last Checkpoint: 09/25/2026 (autosaved)

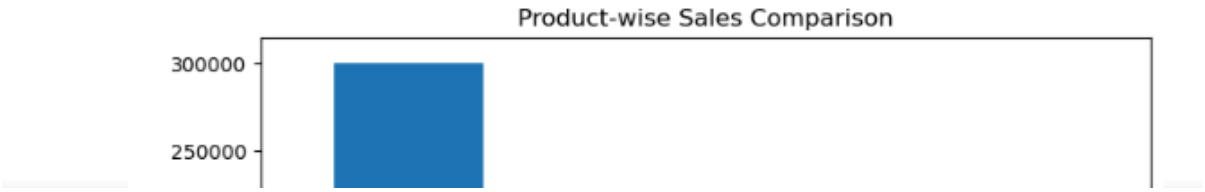
Edit View Insert Cell Kernel Widgets Help

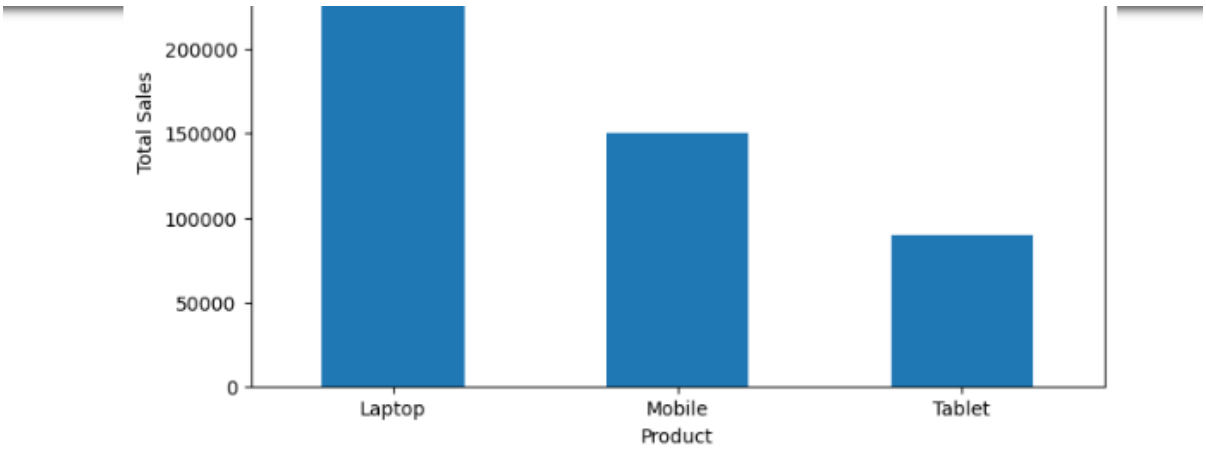
Run Code

```
product_sales.plot(
kind="bar"
)
plt.title("Product-wise Sales Comparison")
plt.xlabel("Product")
plt.ylabel("Total Sales")
plt.xticks(rotation=0)
plt.show()
```




```
Product-wise Sales:
Product
Laptop    300000
Mobile    150000
Tablet     90000
Name: Sales, dtype: int64
```





AIM:

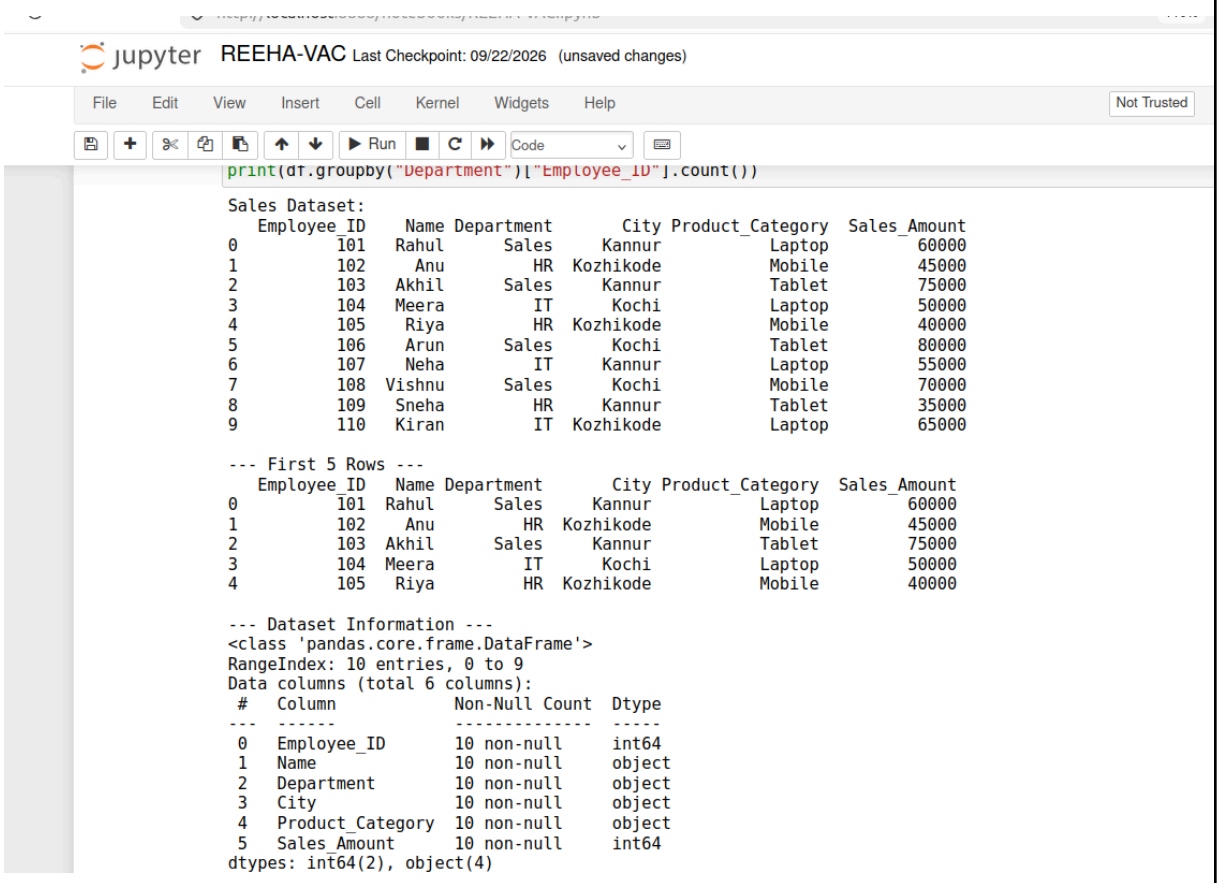
A company maintains sales records containing employee details, employee id name department,city, product category and sales amount.Create the DataFrame, Display the first 5 rows, Display dataset information, Find total sales amount, Group data department-wise. Find department-wise total sales. Find department-wise average sales. Find city-wise maximum sales. Find category-wise minimum sales. Find count of employees in each department.

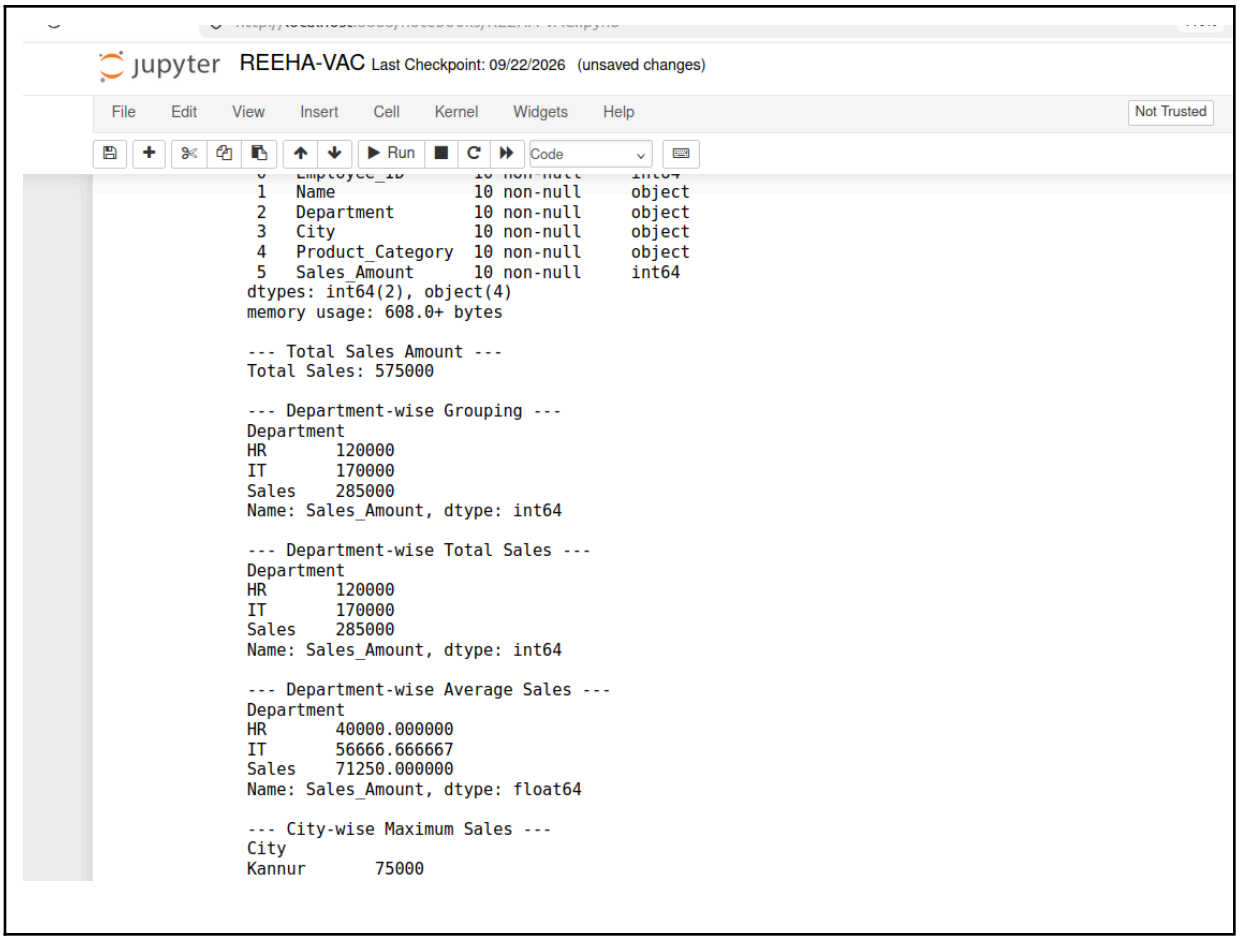
PROGRAM:

```
import pandas as pd
data = {
"Employee_ID": [101, 102, 103, 104, 105, 106, 107, 108, 109, 110],
"Name": ["Rahul", "Anu", "Akhil", "Meera", "Riya",
"Arun", "Neha", "Vishnu", "Sneha", "Kiran"],
"Department": ["Sales", "HR", "Sales", "IT", "HR",
"Sales", "IT", "Sales", "HR", "IT"],
"City": ["Kannur", "Kozhikode", "Kannur", "Kochi", "Kozhikode",
"Kochi", "Kannur", "Kochi", "Kannur", "Kozhikode"],
"Product_Category": ["Laptop", "Mobile", "Tablet", "Laptop", "Mobile",
"Tablet", "Laptop", "Mobile", "Tablet", "Laptop"],
"Sales_Amount": [60000, 45000, 75000, 50000, 40000,
80000, 55000, 70000, 35000, 65000]
}
df = pd.DataFrame(data)
print("Sales Dataset:")
print(df)
print("\n--- First 5 Rows ---")
print(df.head(5))
```

```
print("\n--- Dataset Information ---")
df.info()
print("\n--- Total Sales Amount ---")
print("Total Sales:", df["Sales_Amount"].sum())
print("\n--- Department-wise Grouping ---")
grouped = df.groupby("Department")
print(grouped["Sales_Amount"].sum())
print("\n--- Department-wise Total Sales ---")
print(df.groupby("Department")["Sales_Amount"].sum())
print("\n--- Department-wise Average Sales ---")
print(df.groupby("Department")["Sales_Amount"].mean())
print("\n--- City-wise Maximum Sales ---")
print(df.groupby("City")["Sales_Amount"].max())
print("\n--- Category-wise Minimum Sales ---")
print(df.groupby("Product_Category")["Sales_Amount"].min())
print("\n--- Employee Count in Each Department ---")
print(df.groupby("Department")["Employee_ID"].count())
```

OUTPUT:




jupyter REEHA-VAC Last Checkpoint: 09/22/2026 (unsaved changes)
File Edit View Insert Cell Kernel Widgets Help
Not Trusted
Run
Code
1 Name 10 non-null object
2 Department 10 non-null object
3 City 10 non-null object
4 Product_Category 10 non-null object
5 Sales_Amount 10 non-null int64
dtypes: int64(2), object(4)
memory usage: 608.0+ bytes
--- Total Sales Amount ---
Total Sales: 575000
--- Department-wise Grouping ---
Department
HR 120000
IT 170000
Sales 285000
Name: Sales_Amount, dtype: int64
--- Department-wise Total Sales ---
Department
HR 120000
IT 170000
Sales 285000
Name: Sales_Amount, dtype: int64
--- Department-wise Average Sales ---
Department
HR 40000.000000
IT 56666.666667
Sales 71250.000000
Name: Sales_Amount, dtype: float64
--- City-wise Maximum Sales ---
City
Kannur 75000

jupyter REEHA-VAC Last Checkpoint: 09/22/2026 (unsaved changes)

File Edit View Insert Cell Kernel Widgets Help Not Trusted

Run Code

```
Department
HR       120000
IT       170000
Sales    285000
Name: Sales_Amount, dtype: int64

--- Department-wise Average Sales ---
Department
HR       40000.000000
IT       56666.666667
Sales    71250.000000
Name: Sales_Amount, dtype: float64

--- City-wise Maximum Sales ---
City
Kannur       75000
Kochi        80000
Kozhikode    65000
Name: Sales_Amount, dtype: int64

--- Category-wise Minimum Sales ---
Product_Category
Laptop    50000
Mobile    40000
Tablet    35000
Name: Sales_Amount, dtype: int64

--- Employee Count in Each Department ---
Department
HR       3
IT       3
Sales    4
Name: Employee_ID, dtype: int64
```

In [ ]:

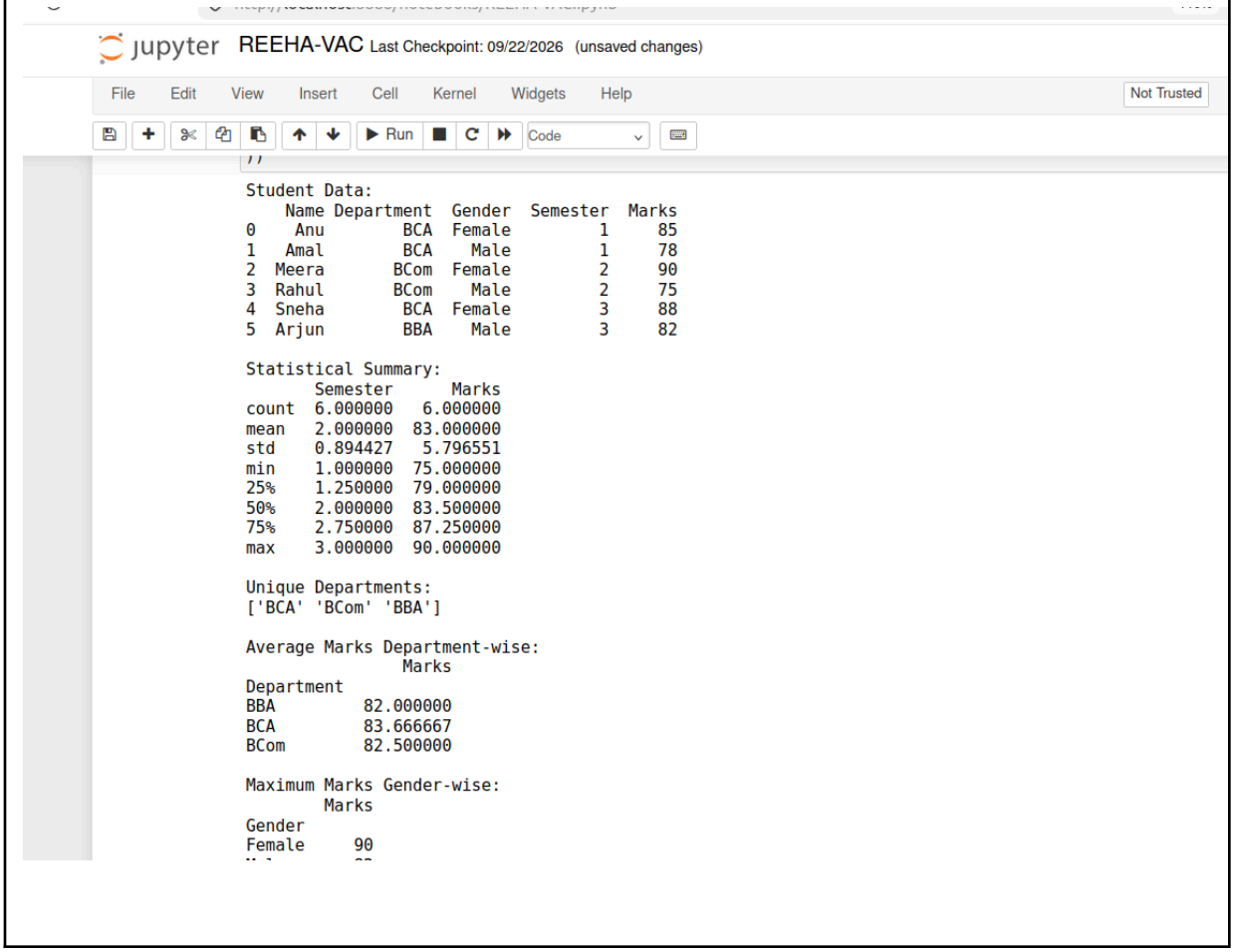
AIM:

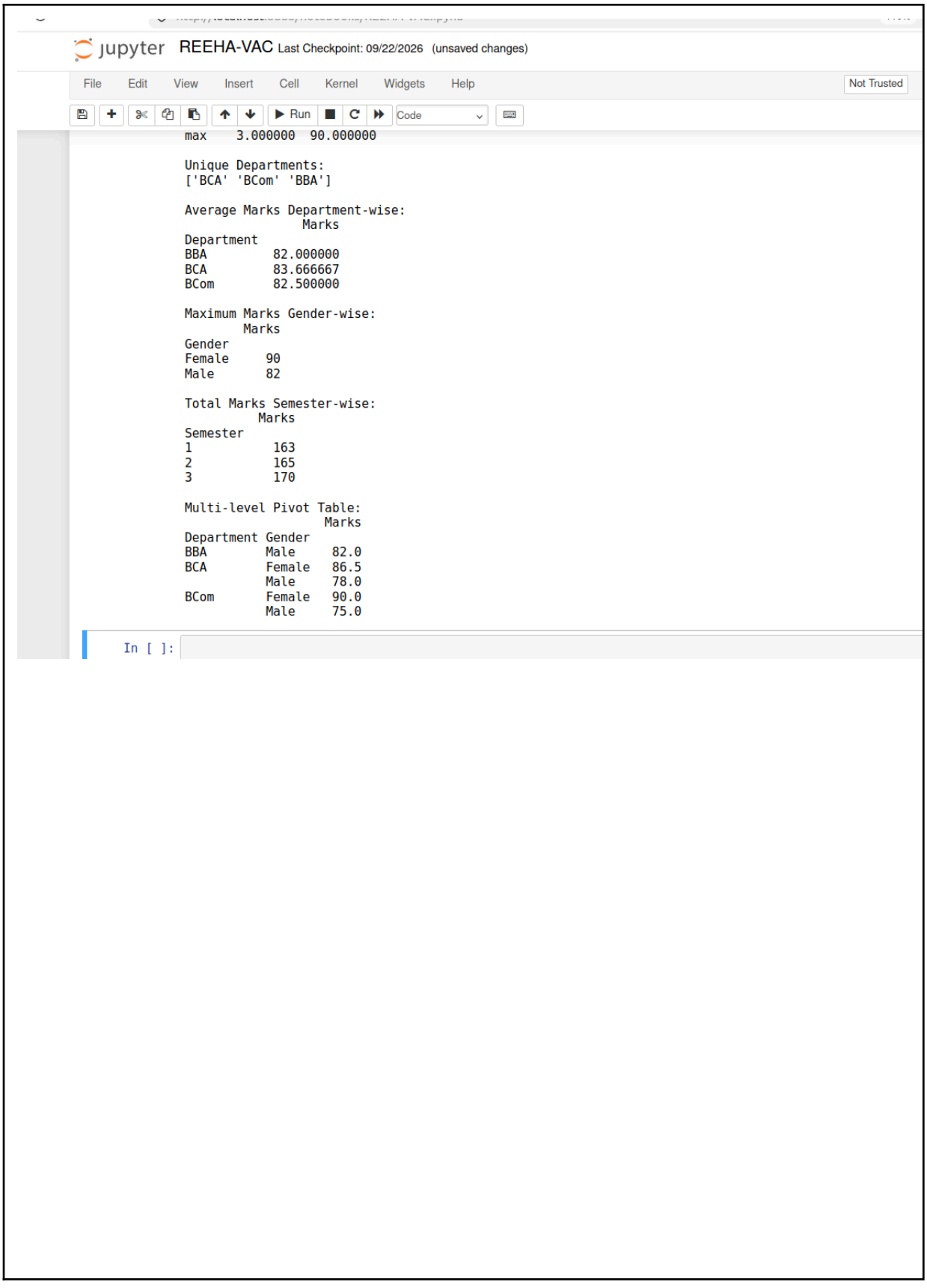
An institute maintains student records containing Name, department, gender, semester, and marks. Create the DataFrame.Display statistical summary. Display unique departments. Create a pivot table showing average marks department-wise. Create a pivot table showing gender-wise maximum marks. Create a pivot table showing semester-wise total marks. Create a multi-level pivot table using Department and Gender.

PROGRAM:

```
import pandas as pd
data = {
"Name": ["Anu", "Amal", "Meera", "Rahul", "Sneha", "Arjun"],
"Department": ["BCA", "BCA", "BCom", "BCom", "BCA", "BBA"],
"Gender": ["Female", "Male", "Female", "Male", "Female", "Male"],
"Semester": [1, 1, 2, 2, 3, 3],
"Marks": [85, 78, 90, 75, 88, 82]
}
df = pd.DataFrame(data)
print("Student Data:")
print(df)
print("\nStatistical Summary:")
print(df.describe())
print("\nUnique Departments:")
print(df["Department"].unique())
print("\nAverage Marks Department-wise:")
print(df.pivot_table(
values="Marks",
index="Department",
aggfunc="mean"
))
print("\nMaximum Marks Gender-wise:")
print(df.pivot_table(
values="Marks",
index="Gender",
aggfunc="max"
))
print("\nTotal Marks Semester-wise:")
print(df.pivot_table(
values="Marks",
index="Semester",
aggfunc="sum"
))
print("\nMulti-level Pivot Table:")
print(df.pivot_table(
values="Marks",
index=["Department", "Gender"],
aggfunc="mean"
))
```

OUTPUT:


jupyter REEHA-VAC Last Checkpoint: 09/22/2026 (unsaved changes)
File Edit View Insert Cell Kernel Widgets Help
Not Trusted
Run
Code
'''
Student Data:
Name Department Gender Semester Marks
0 Anu BCA Female 1 85
1 Amal BCA Male 1 78
2 Meera BCom Female 2 90
3 Rahul BCom Male 2 75
4 Sneha BCA Female 3 88
5 Arjun BBA Male 3 82
Statistical Summary:
Semester Marks
count 6.000000 6.000000
mean 2.000000 83.000000
std 0.894427 5.796551
min 1.000000 75.000000
25% 1.250000 79.000000
50% 2.000000 83.500000
75% 2.750000 87.250000
max 3.000000 90.000000
Unique Departments:
['BCA' 'BCom' 'BBA']
Average Marks Department-wise:
Marks
Department
BBA 82.000000
BCA 83.666667
BCom 82.500000
Maximum Marks Gender-wise:
Marks
Gender
Female 90

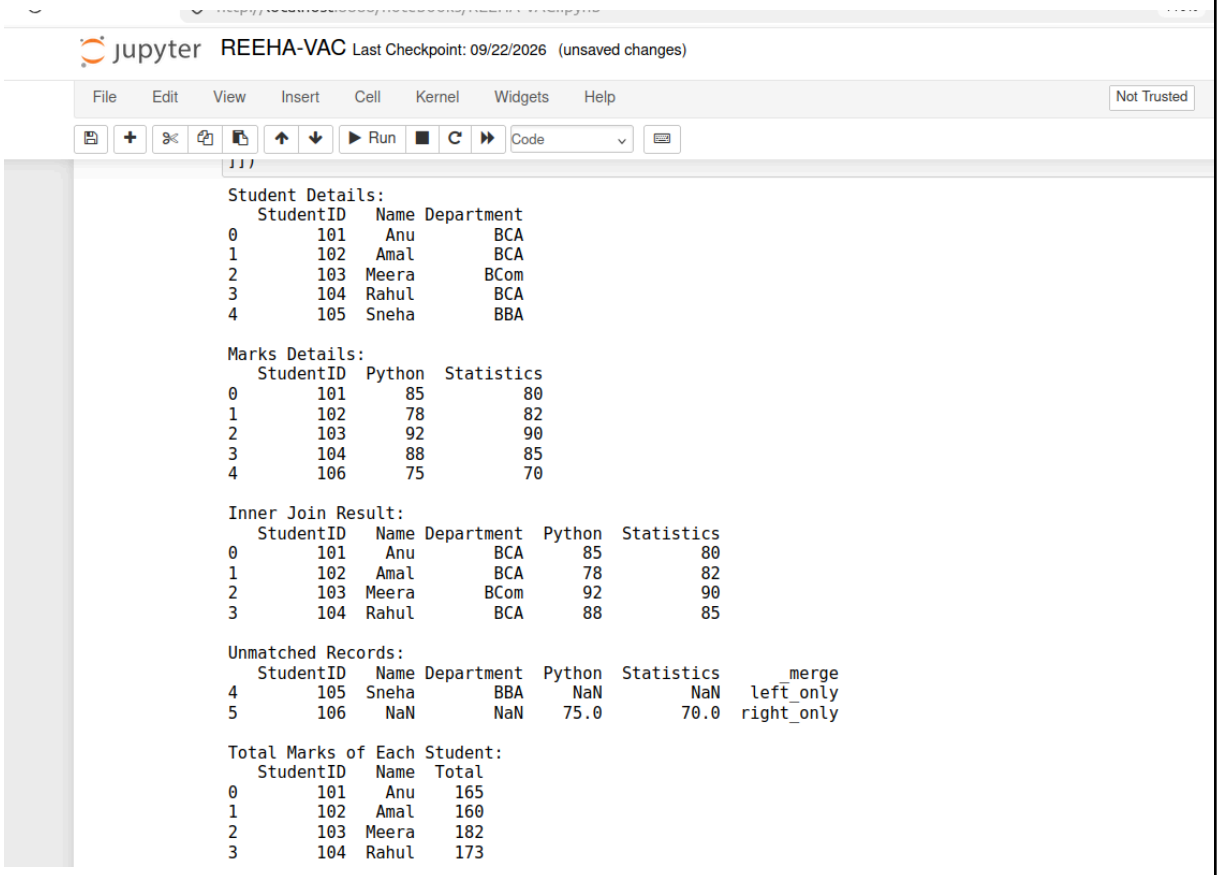

jupyter REEHA-VAC Last Checkpoint: 09/22/2026 (unsaved changes)
File Edit View Insert Cell Kernel Widgets Help
Not Trusted
Run
Code
max 3.000000 90.000000
Unique Departments:
['BCA' 'BCom' 'BBA']
Average Marks Department-wise:
Marks
Department
BBA 82.000000
BCA 83.666667
BCom 82.500000
Maximum Marks Gender-wise:
Marks
Gender
Female 90
Male 82
Total Marks Semester-wise:
Marks
Semester
1 163
2 165
3 170
Multi-level Pivot Table:
Marks
Department Gender
BBA Male 82.0
BCA Female 86.5
Male 78.0
BCom Female 90.0
Male 75.0
In [ ]:

AIM:
A college maintains student details(Student ID, Name Department) in one dataset and marks details (StudentID, Python, Statistics) in another dataset. Create both DataFrames. Display both datasets. Perform inner join. Identify unmatched records after merging. Calculate total marks of each student. Find average marks. Find topper in Python. Find department-wise average marks. Sort students based on total marks.

PROGRAM:

```
import pandas as pd
student_data = {
"StudentID": [101, 102, 103, 104, 105],
"Name": ["Anu", "Amal", "Meera", "Rahul", "Sneha"],
"Department": ["BCA", "BCA", "BCom", "BCA", "BBA"]
}
students = pd.DataFrame(student_data)
marks_data = {
"StudentID": [101, 102, 103, 104, 106],
"Python": [85, 78, 92, 88, 75],
"Statistics": [80, 82, 90, 85, 70]
}
marks = pd.DataFrame(marks_data)
print("Student Details:")
print(students)
print("\nMarks Details:")
print(marks)
merged = pd.merge(students, marks, on="StudentID", how="inner")
print("\nInner Join Result:")
print(merged)
all_data = pd.merge(
students,
marks,
on="StudentID",
how="outer",
indicator=True
)
unmatched = all_data[all_data["_merge"] != "both"]
print("\nUnmatched Records:")
print(unmatched)
merged["Total"] = merged["Python"] + merged["Statistics"]
print("\nTotal Marks of Each Student:")
print(merged[["StudentID", "Name", "Total"]])
merged["Average"] = merged["Total"] / 2
print("\nAverage Marks:")
print(merged[["StudentID", "Name", "Average"]])
topper = merged.loc[merged["Python"].idxmax()]
print("\nPython Topper:")
print(topper["Name"])
print("Python Marks:", topper["Python"])
dept_average = merged.groupby("Department")[["Python", "Statistics"]].mean()
```

```
print("\nDepartment-wise Average Marks:")
print(dept_average)
sorted_students = merged.sort_values(
by="Total",
ascending=False
)
print("\nStudents Sorted by Total Marks:")
print(sorted_students[[
"StudentID",
"Name",
"Department",
"Python",
"Statistics",
"Total",
"Average"
]])
```




```
Student Details:
   StudentID   Name Department
0        101    Anu        BCA
1        102   Amal        BCA
2        103  Meera       BCom
3        104  Rahul        BCA
4        105  Sneha        BBA

Marks Details:
   StudentID  Python  Statistics
0        101      85          80
1        102      78          82
2        103      92          90
3        104      88          85
4        106      75          70

Inner Join Result:
   StudentID   Name Department  Python  Statistics
0        101    Anu        BCA      85          80
1        102   Amal        BCA      78          82
2        103  Meera       BCom      92          90
3        104  Rahul        BCA      88          85

Unmatched Records:
   StudentID   Name Department  Python  Statistics      _merge
4        105  Sneha        BBA     NaN         NaN   left_only
5        106    NaN        NaN    75.0        70.0  right_only

Total Marks of Each Student:
   StudentID   Name  Total
0        101    Anu    165
1        102   Amal    160
2        103  Meera    182
3        104  Rahul    173
```

jupyter REEHA-VAC Last Checkpoint: 09/22/2026 (unsaved changes)

File Edit View Insert Cell Kernel Widgets Help Not Trusted

Run Code

```
Total Marks of Each Student:
   StudentID   Name  Total
0        101    Anu    165
1        102   Amal    160
2        103  Meera    182
3        104  Rahul    173

Average Marks:
   StudentID   Name  Average
0        101    Anu     82.5
1        102   Amal     80.0
2        103  Meera     91.0
3        104  Rahul     86.5

Python Topper:
Meera
Python Marks: 92

Department-wise Average Marks:
                Python  Statistics
Department
BCA          83.666667   82.333333
BCom         92.000000   90.000000

Students Sorted by Total Marks:
   StudentID   Name Department  Python  Statistics  Total  Average
2        103  Meera       BCom      92          90    182     91.0
3        104  Rahul        BCA      88          85    173     86.5
0        101    Anu        BCA      85          80    165     82.5
1        102   Amal        BCA      78          82    160     80.0
```

In [ ]: